Lab: Logistic Regression for Gene Expression Data

In this lab, we use logistic regression to predict biological characteristics ("phenotypes") from gene expression data. In addition to the concepts in breast cancer demo, you will learn to:

- Handle missing data
- Perform multi-class logistic classification
- Create a confusion matrix
- Use L1-regularization for improved estimation in the case of sparse weights (Grad students only)

Background

Genes are the basic unit in the DNA and encode blueprints for proteins. When proteins are synthesized from a gene, the gene is said to "express". Micro-arrays are devices that measure the expression levels of large numbers of genes in parallel. By finding correlations between expression levels and phenotypes, scientists can identify possible genetic markers for biological characteristics.

The data in this lab comes from:

https://archive.ics.uci.edu/ml/datasets/Mice+Protein+Expression

In this data, mice were characterized by three properties:

- Whether they had down's syndrome (trisomy) or not
- Whether they were stimulated to learn or not
- Whether they had a drug memantine or a saline control solution.

With these three choices, there are 8 possible classes for each mouse. For each mouse, the expression levels were measured across 77 genes. We will see if the characteristics can be predicted from the gene expression levels. This classification could reveal which genes are potentially involved in Down's syndrome and if drugs and learning have any noticeable effects.

Load the Data

We begin by loading the standard modules.

```
import pandas as pd
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn import linear_model, preprocessing
```

Use the `pd.read_excel` command to read the data from

https://archive.ics.uci.edu/ml/machine-learning-databases/00342/Data_Cortex_Nuclear.xls

into a dataframe `df`. Use the `index_col` option to specify that column 0 is the index. Use the `df.head()` to print the first few rows.

```
# TODO 1
#   df = pd.read_excel(...)
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00342/Data_Cortex_Nuclear.
df = pd.read_excel(url, index_col=0)
df.head()
```

MouseID	DYRK1A_N	ITSN1_N	BDNF_N	NR1_N	NR2A_N	pAKT_N	pBRAF_N	pCAMKII_N	pCR
309_1	0.503644	0.747193	0.430175	2.816329	5.990152	0.218830	0.177565	2.373744	0.23
309_2	0.514617	0.689064	0.411770	2.789514	5.685038	0.211636	0.172817	2.292150	0.22
309_3	0.509183	0.730247	0.418309	2.687201	5.622059	0.209011	0.175722	2.283337	0.23
309_4	0.442107	0.617076	0.358626	2.466947	4.979503	0.222886	0.176463	2.152301	0.20
309_5	0.434940	0.617430	0.358802	2.365785	4.718679	0.213106	0.173627	2.134014	0.19

5 rows × 81 columns

This data has missing values. The site:

http://pandas.pydata.org/pandas-docs/stable/missing_data.html

has an excellent summary of methods to deal with missing values. Following the techniques there, create a new data frame `df1` where the missing values in each column are filled with the mean values from the non-missing values.

```
# TODO 2
#  df1 = ...

df1 = df.fillna(df.mean(numeric_only=True))
```

Binary Classification for Down's Syndrome

We will first predict the binary class label in `df1['Genotype']` which indicates if the mouse has Down's syndrome or not. Get the string values in `df1['Genotype'].values` and convert this to a

numeric vector `y` with 0 or 1. You may wish to use the `np.unique` command with the `return_inverse=True` option.

```
# TODO 3
#   y = ...

y = np.unique(df1['Genotype'].values, return_inverse=True)
```

As predictors, get all but the last four columns of the dataframes. Store the data matrix into `X` and the names of the columns in `xnames`.

```
# TODO 4
#   xnames = ...
#   X = ...

xnames = df1.columns[:-4]
X = df1[xnames].values
```

Split the data into training and test with 30% allocated for test. You can use the train

```
from sklearn.model_selection import train_test_split

#Use : shuffle=True, random_state=3 so we all can have same split.
# TODO 5:
#   Xtr, Xts, ytr, yts = ...

Xtr, Xts, ytr, yts = train_test_split(X, y[1], test_size=0.3, shuffle=True, random_state=
```

Scale the data with the `StandardScaler`. Store the scaled values in `Xtr1` and `Xts1`.

```
from sklearn.preprocessing import StandardScaler

# TODO 6
#  Xtr1 = ...
#  Xts1 = ...

scaler = StandardScaler()
scaler.fit(Xtr)
Xtr1 = scaler.transform(Xtr)
Xts1 = scaler.transform(Xts)
```

Create a `LogisticRegression` object `logreg` and `fit` on the scaled training data. Set the regularization level to `C=1e5` and use the optimizer `solver=liblinear`.

```
# TODO 7
#   logreg = ...
logreg = linear_model.LogisticRegression(C=1e5, solver='liblinear')
logreg.fit(Xtr1, ytr)
```

```
LogisticRegression
LogisticRegression(C=100000.0, solver='liblinear')
```

Measure the accuracy of the classifer on test data. You should get around 94%.

```
# TODO 8
#   yhat = ...

yhat = logreg.predict(Xts1)
print(np.mean(yhat == yts))
```

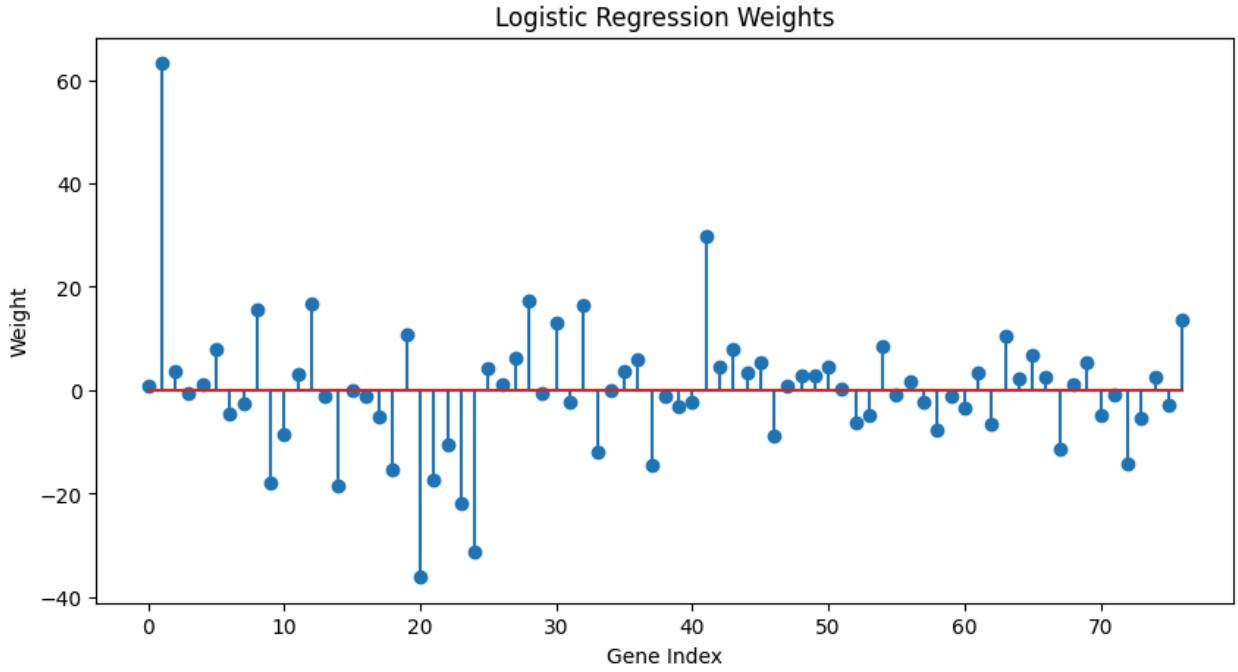
```
0.9506172839506173
```

Interpreting the weight vector

Create a stem plot of the coefficients, `W` in the logistic regression model. Jse the `plt.stem()` function with the `use_line_collection=True` option. You can get the coefficients from `logreg.coef_`, but you will need to reshape this to a 1D array.

```
# TODO 9
#   W = ...
#   plt.stem(...)

W = logreg.coef_.reshape(-1)
plt.figure(figsize=(10,5))
plt.stem(W)
plt.title("Logistic Regression Weights")
plt.xlabel("Gene Index")
plt.ylabel("Weight")
plt.show()
```



You should see that `W[i]` is very large for a few components `i`. These are the genes that are likely to be most involved in Down's Syndrome. Below we will use L1 regression to enforce sparsity. Find the names of the genes for two components `i` where the magnitude of `W[i]` is largest.

```
# TODO 10
top_indices = np.argsort(np.abs(W))[-2:]
print(xnames[top_indices])
```

```
Index(['BRAF_N', 'ITSN1_N'], dtype='object')
```

Cross Validation

To obtain a slightly more accurate result, now perform 10-fold cross validation and measure the average precision, recall and f1-score. Note, that in performing the cross-validation, you will want to randomly permute the test and training sets using the `shuffle` option. In this data set, all the samples from each class are bunched together, so shuffling is essential. Print the mean precision, recall and f1-score and error rate across all the folds.

```
from sklearn.model_selection import KFold
from sklearn.metrics import precision_recall_fscore_support
nfold = 10
kf = KFold(n_splits=nfold,shuffle=True)

# TODO 11

precision = []
recall = []
f1 = []
```

```
err = []

for train, test in kf.split(X):
    Xtr_k, Xts_k = X[train], X[test]
    ytr_k, yts_k = y[1][train], y[1][test]

    scaler = StandardScaler()
    scaler.fit(Xtr_k)
    Xtr1_k = scaler.transform(Xtr_k)
    Xts1_k = scaler.transform(Xts_k)

    logreg_k = linear_model.LogisticRegression(C=1e5, solver='liblinear')
    logreg_k.fit(Xtr1_k, ytr_k)

    yhat_k = logreg_k.predict(Xts1_k)
    p, r, f, s = precision_recall_fscore_support(yts_k, yhat_k, average='binary')
    err_k = np.mean(yhat_k != yts_k)

    precision.append(p)
    recall.append(r)
    f1.append(f)
    err.append(err_k)

print("Precision: %.3f +- %.3f" % (np.mean(precision), np.std(precision)))
print("Recall: %.3f +- %.3f" % (np.mean(recall), np.std(recall)))
print("F1: %.3f +- %.3f" % (np.mean(f1), np.std(f1)))
print("Error: %.3f +- %.3f" % (np.mean(err), np.std(err)))
```

```
Precision: 0.949 +- 0.033
Recall: 0.947 +- 0.030
F1: 0.948 +- 0.022
Error: 0.049 +- 0.020
```

Multi-Class Classification

Now use the response variable in `df1['class']`. This has 8 possible classes. Use the `np.unique` funtion as before to convert this to a vector `y` with values 0 to 7.

```
# TODO 12
#   y = ...
y = np.unique(df1['class'].values, return_inverse=True)
```

Fit a multi-class logistic model by creating a `LogisticRegression` object, `logreg` and then calling the `logreg.fit` method.

Now perform 10-fold cross validation, and measure the confusion matrix `C` on the test data in each fold. You can use the `confustion_matrix` method in the `sklearn` package. Add the confusion matrix counts across all folds and then normalize the rows of the confusion matrix so that they sum to one. Thus, each element `C[i,j]` will represent the fraction of samples where `yhat==j` given `ytrue==i`. Print the confusion matrix. You can use the command

```
print(np.array_str(C, precision=4, suppress_small=True))
```

to create a nicely formatted print. Also print the overall mean and SE of the test accuracy across the folds.

```
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import KFold

# TODO 13

C_total = np.zeros((8, 8))
acc_list = []

for train, test in kf.split(X):
    Xtr_k, Xts_k = X[train], X[test]
    ytr_k, yts_k = y[1][train], y[1][test]

    scaler_k = StandardScaler()
    Xtr_k_scaled = scaler_k.fit_transform(Xtr_k)
    Xts_k_scaled = scaler_k.transform(Xts_k)



    logreg_multi = linear_model.LogisticRegression(C=1e5, solver='liblinear')
    logreg_multi.fit(Xtr_k_scaled, ytr_k)


    yhat_k = logreg_multi.predict(Xts_k_scaled)
    C_total += confusion_matrix(yts_k, yhat_k, labels=range(8))
    acc_list.append(np.mean(yhat_k == yts_k))

C_normalized = C_total / C_total.sum(axis=1, keepdims=True)

print(np.array_str(C_normalized, precision=4, suppress_small=True))

print(f"Mean Test Accuracy: {np.mean(acc_list):.4f}")
print(f"SE of Test Accuracy: {np.std(acc_list)/np.sqrt(nfold):.4f}")
```

```
[[0.9733 0.02   0.     0.     0.     0.0067 0.     0.    ]
 [0.0148 0.9778 0.     0.     0.     0.     0.0074 0.    ]
 [0.     0.0067 0.9867 0.0067 0.     0.     0.     0.    ]
 [0.0148 0.     0.     0.9852 0.     0.     0.     0.    ]
 [0.     0.0074 0.     0.     0.9926 0.     0.     0.    ]
 [0.     0.     0.     0.     0.     1.     0.     0.    ]
 [0.     0.     0.     0.0074 0.     0.     0.9926 0.    ]
 [0.     0.     0.     0.     0.     0.     0.     1.    ]]
Mean Test Accuracy: 0.9880
SE of Test Accuracy: 0.0037
```

Re-run the logistic regression on the entire training data and get the weight coefficients. This should be a 8 x 77 matrix. Create a stem plot of the first row of this matrix to see the coefficients on each of the genes.

```
# TODO 14
```

```
scaler_full = StandardScaler()
X_scaled = scaler_full.fit_transform(X)

logreg_multi_full = linear_model.LogisticRegression(C=1e5, solver='liblinear')
logreg_multi_full.fit(X_scaled, y[1])

W_multi = logreg_multi_full.coef_


plt.figure(figsize=(10, 5))
plt.stem(W_multi[0, :])
plt.title("Multi-Class Logistic Regression Weights")
plt.xlabel("Gene Index")
plt.ylabel("Weight")
plt.show()
```



L1-Regularization

This section is bonus.

In most genetic problems, only a limited number of the tested genes are likely influence any particular attribute. Hence, we would expect that the weight coefficients in the logistic regression model should be sparse. That is, they should be zero on any gene that plays no role in the particular attribute of interest. Genetic analysis commonly imposes sparsity by adding an l1-penalty term. Read the `sklearn` documentation on the `LogisticRegression` class to see how to set the l1-penalty and the inverse regularization strength, `C`.

Using the model selection strategies from the housing demo, use K-fold cross validation to select an appropriate inverse regularization strength.

- Use 10-fold cross validation
- You should select around 20 values of `C`. It is up to you find a good range.
- Make appropriate plots and print out to display your results
- How does the accuracy compare to the accuracy achieved without regularization.

```
# TODO 15

C_values = np.logspace(-3, 3, 20)
mean_accs = []

for C_val in C_values:
    acc_fold = []
    for train, test in kf.split(X):
        Xtr_k, Xts_k = X[train], X[test]
        ytr_k, yts_k = y[1][train], y[1][test]

        scaler_k = StandardScaler()
        Xtr_k_scaled = scaler_k.fit_transform(Xtr_k)
        Xts_k_scaled = scaler_k.transform(Xts_k)

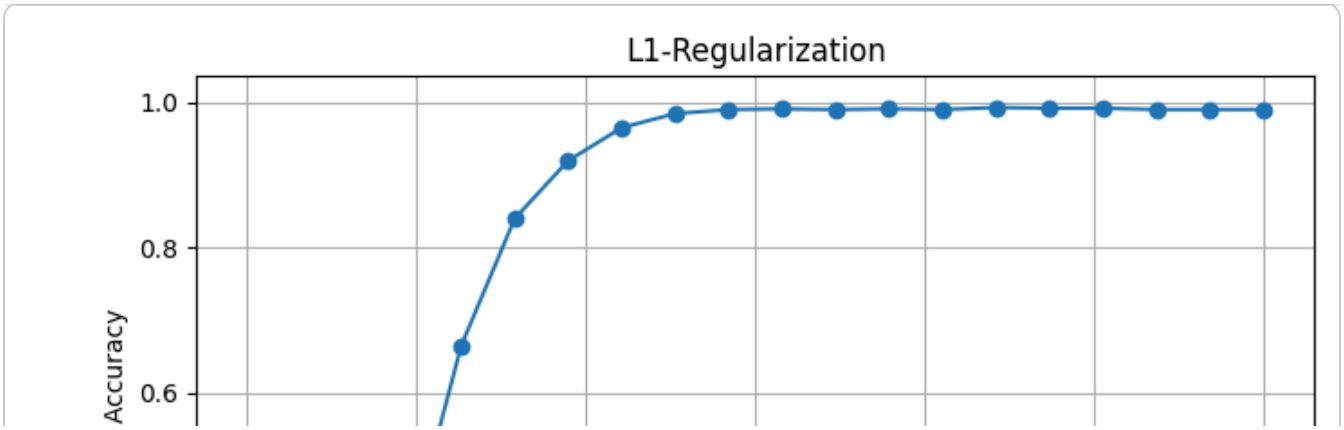
        logreg_l1 = linear_model.LogisticRegression(penalty='l1', C=C_val, solver='liblinea
        logreg_l1.fit(Xtr_k_scaled, ytr_k)

        yhat_k = logreg_l1.predict(Xts_k_scaled)
        acc_fold.append(np.mean(yhat_k == yts_k))

    mean_accs.append(np.mean(acc_fold))

plt.figure(figsize=(8, 5))
plt.semilogx(C_values, mean_accs, marker='o')
plt.xlabel("Inverse Regularization Strength")
plt.ylabel("Mean CV Accuracy")
plt.title("L1-Regularization")
plt.grid(True)
plt.show()

best_idx = np.argmax(mean_accs)
print(f"C value: {C_values[best_idx]:.4f}")
print(f"Accuracy with L1 regularization: {mean_accs[best_idx]:.4f}")
```


L1-Regularization
1.0
0.8
0.6
Accuracy